Obligatory assignment 2 MSA102/MVE188, autumn 2025

Petter Mostad

October 2, 2025

The country Xanadu is holding an election to its parliament. Your task below is to make code that can be used during an election night TV show to make predictions about the final outcome, based on the partial results that have come in so far.

In Xanadu, there are only two parties, A and B. There are $N = 231$ constituencies, each of which sends to the parliament a single representative from the party getting the most votes in that constituency.

At a time t of the election night, you have the following sources of data:

1. For each constituency i, the results of the previous election, expressed as the log-odds q_i of the proportion of votes that party A received[1].

2. Exit poll results: For each of 50 constituencies, $n = 100$ people have been asked which party they voted for as they exited the voting. For a constituency i with an exit poll you have available y_i, the number of persons replying that they voted for A.

3. For constituencies i where the results have come in by time t, you have p_i, the log-odds of the proportion of votes that party A received.

Your task is to make code that computes, given the data above, the probability that party A will win a majority of the representatives.

The data is stored in the file "election25.txt", which you may read in with the command `read.table("election25.txt",header=TRUE)`. The first column is q_i, the log-odds results from the previous election. The second column lists exit poll results, i.e., the count y_i of those who voted for party A among 100 people asked in constituency i. For constituencies with no exit polls, the results are indicated as NA. The third column lists t_i, the time that the final results are received for constituency i, measured in hours after voting closed. The fourth column lists p_i, the log-odds of the final results from the constituency.

[1] If a proportion is r then the log-odds is $\log(r/(1-r))$.

For each i we write $d_i = p_i - q_i$. We assume that, for each i independently,

$$d_i \mid \mu, \gamma \sim \text{Normal}(\mu, e^{-\gamma})$$

for parameters μ and γ, for which we assume flat priors. Further, in the constituencies with exit poll results, we assume that

$$y_i \mid d_i \sim \text{Binomial}\left(n, \frac{e^{d_i+q_i}}{1+e^{d_i+q_i}}\right).$$

To facilitate the theory and coding below, we also introduce the following notation to be used at a fixed time t: Assume that, at time t, s constituencies have final results, r constituencies have only poll data results, and $N-s-r$ constituencies have no results. Let ω denote the vector $\omega_1, \ldots, \omega_s$ representing the d_i values for constituencies with final results, let ψ denote the vector $\psi_1, \ldots, \psi_r$ representing the d_i values for constituencies with only poll data results, and let θ denote the vector $\theta_1, \ldots, \theta_{N-r-s}$ representing the d_i of constituencies with no data at time t. For the r constituencies with only poll data, we use $z_1, \ldots, z_r$ to denote these poll results and $w_1, \ldots, w_r$ to denote their results from the previous election (i.e., the z_i's are the y_i's re-indexed, and the w_i's are the q_i's re-indexed).

1. (a) Read in and visualize the data. Draw a Bayesian Network of the model at time t.

   (b) Considering the Bayesian network, prove that

   $$\pi(\theta, \psi \mid \omega, z) = \iint \pi(\theta \mid \mu, \gamma)\pi(\mu, \gamma, \psi \mid \omega, z)\, d\mu\, d\gamma.$$

   (c) Write down a formula for $U_t(\mu, \gamma, \psi)$, the negative logarithm of the posterior density (up to a constant) for the variables μ, γ, ψ given the observed data at time t. Write down a formula for $\nabla U_t(\mu, \gamma, \psi)$, the gradient of $U_t(\mu, \gamma, \psi)$.

   (d) Program a Hamiltonian Monte Carlo algorithm producing an approximate sample from the posterior of (μ, γ, ψ) given data at time t: Write a function that takes as input the data and the parameter t. It extracts the variables at time t as defined above, and defines a function "U" taking as a single input a vector q consisting of μ, γ, and ψ and producint as output $U_t(\mu, \gamma, \psi)$. Also, create a function "gradU" taking as a single input the vector q consisting of μ, γ, and ψ and giving as output $\nabla U_t(\mu, \gamma, \psi)$. (Make sure you do some small tests showing that "gradU" is indeed the gradient of "U".) Finally, implement the Hamiltonian Monte Carlo algorithm (you may re-use example code used in the lecture about HMC). Test your function for $t = 3.4$ and visualize its output.

   (e) Extend your function from (d) to instead return the probability (approximated with Monte Carlo integration) at time t that party A will win a majority of the seats in the election.

(f) Make a plot which, at each time new data arrives, plots the *proportion* of the *determined* seats that party A has won so far, together with the probability that party A will win the election. Make some comments to explain the plot.

(g) The computation implemented above may be a bit inefficient, as we start "from scratch" at each new time point.

- An alternative is to store the posterior for (μ, γ, ψ) given the information available so far, and then update this posterior whenever the information changes. Write down a formula for the posterior and describe exactly how the posterior changes when results come in from a constituency without poll data, and when results come in from a constituency with poll data.
- Another alternative is to instead store the marginal posterior for (μ, γ) given information available so far. Describe exactly how this posterior changes when results come in from a constituency without poll data, and when results come in from a constituency with poll data.
- Do you have ideas of how to use the above in an algorithm?